The Bugzilla Master Guide — in Plain English

A complete, beginner-friendly guide to Bugzilla — plain-English edition

Prepared 17 October 2026

Contents

Before you start 3
Section 1: What is Bugzilla & why use it? 3
What is a bug? 3
What is a bug tracker? 3
Why do teams need one? 4
What is Bugzilla? 4
Why use Bugzilla? The main benefits 4
Bugzilla vs other tools 5
Pros & cons 5
Section 2: Core Bugzilla concepts, step by step 7
1. Products 7
2. Components 7
3. Versions & milestones 7
4. Users & roles 8
5. Groups & permissions 8
Section 3: The life of a bug 10
Step 1 — New / Unconfirmed: a problem is found and reported 10
Step 2 — Confirmed, assigned, in progress: someone takes charge 10
Step 3 — Resolved: the developer submits the fix 10
Step 4 — Verified / Closed: QA re-tests and confirms 11
Section 4: Complete feature & configuration guide 12
4.1 How to create products and attach components 12
4.2 How to add users and assign roles 13
4.3 How to create groups and map users and products to them 14
4.4 How to file a bug 15
4.5 How to search and filter bugs 17
4.6 How email notifications work 18
4.7 How to generate reports and graphs 19
Quick reference 20

A complete, beginner-friendly guide to Bugzilla: what it is, how it works, and how to use every main feature. No technical background needed.

Before you start

- This guide is about **Bugzilla itself**, the free, open-source bug tracker used by projects like Mozilla Firefox and the Linux kernel.
- Every technical word is explained the first time it appears, with an everyday example.
- Words in **bold** are things you will see on the screen. Words in `code` are exact names or values.
- Bugzilla version 5 is described. Older versions look slightly different, but the ideas are the same.

Section 1: What is Bugzilla & why use it?

What is a bug?

Imagine you buy a toy robot. The box says it walks and talks. You press the button and it walks, but it does not talk. **Something is not working the way it was promised.** In software, that is called a **bug**.

A bug can be big or small:

- A banking app crashes when you tap "Pay". *(Big.)*
- A spelling mistake on a button. *(Small.)*
- A page that takes 20 seconds to load. *(Annoying.)*
- A security hole that lets strangers see your data. *(Dangerous.)*

What is a bug tracker?

Picture a classroom with 30 children and one teacher. If every child shouts a problem at the same time, the teacher remembers none of them. So the teacher keeps a **notebook**: one line per problem, who reported it, who is fixing it, and whether it is done.

A **bug tracker** is that notebook for a software team, but shared, searchable and impossible to lose. Every problem becomes a **ticket** (also called a **bug report** or just **a bug**) with:

- a **number** (so everyone can refer to "bug 1234"),
- a **title** (one line: what is wrong),
- a **description** (how to see the problem yourself),
- a **status** (new? being fixed? done?),
- an **owner** (the one person responsible right now),
- a **history** (every change, by whom, and when).

Why do teams need one?

- **Nothing gets forgotten.** A problem mentioned in a hallway disappears. A ticket stays until it is truly fixed.
- **No arguing about who is doing what.** Every ticket has exactly one owner.
- **Everyone sees the same truth.** Managers, testers and developers look at the same list.
- **You can measure.** How many bugs are open? How many are serious? Are we getting better or worse?
- **The past is kept.** Months later you can see why a decision was made.

What is Bugzilla?

Bugzilla is one of the oldest and most trusted bug trackers in the world. It was created in 1998 by the people who make the Firefox browser, and it is **free** and **open source** (meaning anyone can use it and read how it is built, at no cost).

Think of Bugzilla as a **very organised filing cabinet** for problems:

- Each **drawer** is a product (the app or system you are testing).
- Each **folder** inside a drawer is a component (a part of that product).
- Each **sheet of paper** in a folder is one bug.
- A **front desk clerk** (the status workflow) makes sure every sheet moves through the right steps: reported → confirmed → being fixed → fixed → checked.
- A **security guard** (groups and permissions) decides who may open which drawer.

Bugzilla runs on your own server, so your data stays with you. People use it from any web browser; nothing needs to be installed on their computers.

Why use Bugzilla? The main benefits

- **It costs nothing**, for 5 users or 5,000.
- **It is built for bugs**, not adapted from something else. Every field a bug needs is already there.
- **It never forgets.** Every change to every bug is recorded forever, with the person and the time. You cannot secretly edit history.
- **It is strict about who sees what.** You can hide an entire product from people who should not see it.
- **It talks to other software.** Automated tests, scripts and custom screens can create and read bugs through its **API** (a doorway programs use to talk to each other, like a drive-through window for computers).
- **It is proven.** Twenty-five years of daily use by huge projects means the rare, strange situations have already been handled.
- **Powerful searching.** Find "all serious bugs in the login page reported last week that nobody has touched" in a few clicks, and save that search to reuse it.

- **Email that tells you what you care about.** You decide which changes to which bugs should email you.

Bugzilla vs other tools

	Bugzilla	Jira	Trello	GitHub Issues
What it is	A dedicated bug tracker	A big project-management suite that also tracks bugs	A board of sticky-note cards	A simple to-do list attached to code
Price	Free	Paid per user, per month	Free basics, paid extras	Free for open code, paid for teams
Where your data lives	Your own server	Usually the maker's cloud	The maker's cloud	GitHub's cloud
Fields for a bug (severity, priority, version, platform…)	All built in	Built in, very customisable	Only labels	Only labels
History of every change	Always, complete	Yes	Partly	Partly
Hide a product from some people	Yes, built in	Yes	Per board only	Per repository only
Looks and feels	Plain, old-fashioned	Modern but busy	Very simple	Simple
Learning curve	Medium	Steep	Easy	Easy

Why Bugzilla is still a classic, powerful choice: when the job is *tracking defects seriously* — with real fields, strict permissions, complete history, and no monthly bill — Bugzilla does that job as well as anything made since. Tools like Jira do more things; Bugzilla does this one thing thoroughly. Many teams put a modern front end on top of Bugzilla to get the best of both.

Pros & cons

Advantages

- Free and open source — no licence, no vendor lock-in.
- Mature, stable and trusted by the largest software projects.
- Rich, structured bug fields out of the box.

- Complete, tamper-proof history on every bug.
- Fine-grained security: control who can see, file and edit bugs per product.
- Strong search, saved searches, and reports.
- Flexible email notifications per person.
- A full API, so test tools and custom interfaces can plug in.
- Self-hosted: sensitive bug details stay inside your company.

Limitations

- The built-in web pages look dated and can feel crowded.
- Not a project-planning tool: no sprint boards, timelines or roadmaps.
- Some administration is "all or nothing" — a handful of settings are powerful enough that you should read before you click.
- It needs a server and someone to look after it (updates, backups).
- Written in Perl, a language few teams know, so changing Bugzilla's insides is rarely done. (Changing *settings* needs no code at all.)

Section 2: Core Bugzilla concepts, step by step

1. Products

A **Product** is a thing you build or test. It is the top-level drawer in the filing cabinet.

- Examples: **Mobile Banking App**, **Website**, **Customer API**, **Admin Portal**.
- Every bug belongs to exactly one product.
- Products can be **active** (new bugs allowed) or **inactive** (kept for history, no new bugs — for example a retired app).

Analogy: a hospital has departments — Cardiology, Radiology, Pharmacy. A product is a department.

Classifications (optional) are an extra layer *above* products, for companies with many products: for example the classification **Customer Apps** containing the products Mobile Banking App and Website. Most teams leave this feature switched off.

2. Components

A **Component** is a part of a product. It is the folder inside the drawer.

- Examples inside Mobile Banking App: **Login Page**, **Payment Gateway**, **Account Statements**, **Push Notifications**.
- Every bug belongs to exactly one component.
- Each component has a **default assignee** — the person who automatically receives new bugs filed in it. Optionally a **default QA contact** too.
- Components keep bugs with the right specialist: a Payment Gateway bug goes straight to the payments developer.

Analogy: inside Cardiology there are wards — ICU, Outpatients, Surgery. A component is a ward, and each ward has a doctor in charge.

3. Versions & milestones

These two answer different questions:

	Question it answers	Example
Version	"In which release did you **find** the bug?"	*Found in version 2.3*
Target Milestone	"In which release do we plan to **fix** it?"	*Fix it in 2.4*

- Both are lists you define per product (e.g. `1.0`, `2.0`, `2.3`, `unspecified`).
- A **version** is picked when the bug is filed. It helps answer "is this an old bug or a new one?".

- A **milestone** is set by the team when planning. Searching "all bugs with milestone 2.4 that are still open" tells you if release 2.4 is ready.

Analogy: version is the model year of the car that had the fault; milestone is the recall batch in which the garage plans to fix it.

4. Users & roles

Everyone who uses Bugzilla has an **account** (an email address and password). What they can do depends on their **role** — in Bugzilla, a role is simply a set of **groups** the account belongs to (groups are explained next).

Role (everyday name)	Who they are	What they typically do
Reporter	Anyone who finds a problem — a tester, a support person, sometimes a customer	Files bugs, adds comments, attaches screenshots
QA tester	The quality team	Files bugs, **confirms** that new bugs are real, checks fixes, closes bugs
Developer	The people who fix things	Takes bugs, fixes them, marks them resolved
Triager / lead	A senior person	Decides priority, assigns bugs to the right developer, marks duplicates
Administrator	The system owner	Creates products, components, users and groups; changes settings

Key points:

- A bug always has one **Assignee** (the owner). Others can watch it by adding themselves to the **CC list** ("carbon copy" — like being copied on an email).
- The **Reporter** of a bug is recorded forever and can always see their own bug.
- There are no fixed role names in Bugzilla; roles are made by combining groups. The table above is how most teams set it up.

5. Groups & permissions

A **group** is a named list of users. Groups do two jobs.

Job 1 — Permissions ("what can you do?"). Bugzilla comes with built-in groups that each unlock an ability:

Group	What it lets you do
`editbugs`	Edit any field on any bug you can see (without it, you can only edit your own bugs and add comments)
`canconfirm`	Confirm new bugs (move them from UNCONFIRMED to CONFIRMED)
`editcomponents`	Create and edit products, components, versions and milestones
`editusers`	Create and edit user accounts
`creategroups`	Create and edit groups
`editkeywords`	Manage the list of keywords
`tweakparams`	Change Bugzilla's settings (parameters)
`admin`	Everything — the super-user

Job 2 — Visibility ("what can you see?"). You can create your own groups and attach them to products, so that only members can see that product's bugs. This is called a **bug group**.

Analogy: a hospital ID badge. The colour of the badge says what you may do (nurse, doctor, admin). The list of doors it opens says where you may go.

How a product is locked to a group (the "group controls"):

- **Entry** — bugs filed into the product are automatically put in the group.
- **Member control / Other control** — whether people inside / outside the group can see the bugs, with four levels: **NA** (not used), **Shown** (the group is offered as an option), **Default** (bugs go in the group unless someone unticks it), **Mandatory** (bugs are always in the group — the setting that truly hides a product).
- **canedit** — only group members may edit the product's bugs.

A bug that is in **several** groups is visible only to people who are in **all** of them. A bug in **no** group is visible to everyone who can log in.

Section 3: The life of a bug

Every bug moves through a set of **statuses**. Think of them as the stations on a train line. Bugzilla's **workflow** says which station you can travel to from each one.

The standard stations in Bugzilla 5:

```
UNCONFIRMED  →  CONFIRMED  →  IN_PROGRESS  →  RESOLVED  →  VERIFIED
     ↑                                            │
     └──────────────── (reopen if still broken) ──┘
```

(Older versions of Bugzilla used the names NEW, ASSIGNED, REOPENED and CLOSED for the same ideas. Administrators can rename or add statuses; many teams add REOPENED and CLOSED back.)

Step 1 — New / Unconfirmed: a problem is found and reported

- A tester (or anyone) notices the problem and clicks **New → File a Bug**.
- They choose the product and component, write a clear **summary** (title) and a **description** with the steps to see the problem, and attach a screenshot.
- If the reporter is trusted (has `canconfirm`), the bug starts as **CONFIRMED**. Otherwise it starts as **UNCONFIRMED** — "someone says there is a problem; nobody has checked yet".
- The bug lands with the component's **default assignee**.

Analogy: a patient arrives at reception and describes their symptoms. They are in the waiting room.

Step 2 — Confirmed, assigned, in progress: someone takes charge

- A tester or lead reproduces the problem and sets it to **CONFIRMED**: "yes, this is real".
- The lead checks the **priority** and **severity**, and makes sure the **assignee** is the right developer. If not, they reassign it.
- The developer starts work and sets **IN_PROGRESS** (older Bugzilla: ASSIGNED): "I am on it". Everyone can see it is being handled.
- While working, the developer adds comments: what they found, questions for the reporter, and so on.

Analogy: the doctor examines the patient, agrees there is a problem, and begins treatment.

Step 3 — Resolved: the developer submits the fix

- When the fix is done, the developer sets the status to **RESOLVED** and must choose a **resolution** — the one-word reason it is finished:

Resolution	Meaning
FIXED	The problem was corrected.
INVALID	It was not actually a bug (expected behaviour, or a mistake in the report).

Resolution	Meaning
WONTFIX	It is a real issue, but we decided not to fix it.
DUPLICATE	Already reported as another bug (you give that bug's number; the two are linked).
WORKSFORME	We could not make the problem happen.
MOVED	Transferred to another bug database.

- Usually a comment says *what* was changed and *where* (for example "Fixed in build 2.4.1").
- The bug now counts as **closed** in reports, but it is not finished until someone checks.

Analogy: the doctor says "treatment complete" and writes it on the chart.

Step 4 — Verified / Closed: QA re-tests and confirms

- A tester tries the original steps again on the fixed version.
- **It works:** they set **VERIFIED** (and, in setups that have it, **CLOSED**). The bug is finished for good.
- **It still fails:** they **reopen** it — back to CONFIRMED (or REOPENED, if the workflow has it) with a comment explaining what still goes wrong. It returns to the developer.

Analogy: a follow-up appointment. If the patient is well, the file is closed. If not, treatment starts again.

Two more things that happen along the way:

- **Duplicates.** When bug 1500 is marked a DUPLICATE of bug 1200, Bugzilla links them, copies the reporter of 1500 onto the CC list of 1200, and shows "1500 has been marked as a duplicate of this bug" on 1200. Nobody loses track.
- **Dependencies.** A bug can **depend on** another ("we cannot fix the checkout until the login bug is fixed") or **block** another. Bugzilla draws a **dependency tree** so you can see the chain.

Section 4: Complete feature & configuration guide

Everything below is done in Bugzilla's own web pages. Administrators find the setup pages under **Administration** (link at the top or bottom of every page). Everyone else uses the links at the top: **Home, New, Browse, Search, Reports, My Requests, Preferences**.

4.1 How to create products and attach components

Create a product

You need the `editcomponents` permission.

1. Click **Administration** → **Products** → **Add product**.
2. Fill in:
   - **Product** — the name, e.g. `Mobile Banking App`.
   - **Description** — a sentence everyone will understand.
   - **Open for bug entry** — ticked (untick later to retire the product).
   - **Version** — the first version label, e.g. `1.0`.
   - **Default milestone** — usually `---`.
   - **Allow UNCONFIRMED** — tick if you want untrusted reporters' bugs to start as UNCONFIRMED.
3. Click **Add**. You will see "Product Created" and a reminder to add components.

Add components

1. **Administration** → **Products** → click the product → **Edit components** → **Add a new component**.
2. Fill in:
   - **Component** — the name, e.g. `Login Page`.
   - **Description**.
   - **Default assignee** — the email address of the person who should receive new bugs here.
   - **Default QA contact** (optional) — the tester responsible for checking fixes.
   - **Default CC list** (optional) — people always copied on bugs in this component.
3. Click **Add**. Repeat for each part of the product (Payment Gateway, Account Statements…).

Add versions and milestones

- **Edit versions** (on the product page) → **Add a version** → type the label (`2.0`) → **Add**.
- **Edit milestones** → **Add a milestone** → type the label (`2.4`) and a **sort key** (a number for ordering) → **Add**.
- Milestones only show on bugs if the setting `usetargetmilestone` is on (Administration → Parameters → Bug Fields).

Edit, rename, disable, delete

- Everything above can be renamed on the same pages. Versions and milestones can be marked **inactive** so they stop being offered but stay on old bugs.
- A product can only be **deleted** when it has no bugs. Prefer unticking **Open for bug entry** instead, which keeps the history.

4.2 How to add users and assign roles

Create a user

You need the `editusers` permission.

1. **Administration** → **Users** → **Add a new user**.
2. Fill in **Login name** (the email address), **Real name** and a **Password**.
3. Click **Add**. The person can sign in immediately.

Give the user a role (group memberships)

1. **Administration** → **Users** → find the user (search by name or email) → click them.
2. Scroll to **Group access**. Each group has two tick boxes:
   - **Member** — the user is in the group (has the ability).
   - **Bless** — the user may add *other* people to this group (a "team lead" power).
3. Typical setups:
   - **Reporter**: no extra groups (everyone can file bugs and comment).
   - **Developer**: `editbugs`.
   - **QA tester / triager**: `editbugs` + `canconfirm`.
   - **Administrator**: `editusers` + `editcomponents` (+ `admin` for the very few people who run the system).
   - Plus the **bug groups** of the products they may see (section 4.3).
4. Click **Save changes**.

Disable a user (there is no delete)

On the user's page, type a message in **Disable text** (e.g. "Left the company") and save. The person can no longer log in, but their name stays on every bug they touched, so history is preserved. Clear the text to re-enable.

Let people create their own accounts (optional)

Administration → Parameters → User Authentication: `createemailregexp` controls who may self-register (for example `.*@yourcompany\.com$` for company emails only; leave empty to disable self-registration).

Impersonate a user (sudo)

Members of `bz_sudoers` can temporarily act as another user (**Administration → Users → user → Impersonate**) to see exactly what that person sees. Every impersonation is logged.

4.3 How to create groups and map users and products to them

Create a group

You need the `creategroups` permission.

1. **Administration → Groups → Add Group**.
2. Fill in:
   - **New name** — e.g. `Payments Team`.
   - **New description** — what it is for.
   - **User RegExp** — leave empty (an advanced option that adds people automatically by email pattern).
   - **Use for bugs** — tick this if the group will hide bugs (a **bug group**). Leave unticked for pure permission groups.
   - **Icon URL** — optional.
3. Click **Add**.

Put people in the group

- **Administration → Users → user → Group access → tick Member → Save**, or
- **Administration → Groups → group → Edit** → under **Groups that are a member of this group** you can also make one group automatically include another (for example everyone in `Admins` is also in `Payments Team`).

Lock a product to the group

1. **Administration → Products → product → Edit Group Access Controls**.
2. Find your group in the list and set:
   - **Entry** — ticked, so new bugs go into the group automatically.
   - **Member control** — **Mandatory**.
   - **Other control** — **Mandatory**.
   - **canedit** — tick if only members may edit the bugs.
3. Click **Update**. From now on, only members of `Payments Team` can see that product's bugs.

Restrict a single bug

On any bug, the **Groups** section (shown when the product has group controls set to Shown or Default) has tick boxes: tick a group to limit the bug to its members. Two extra boxes let the **reporter** and the **CC list** always see it even if they are not in the group.

Keep in mind

- The **reporter** of a bug can always see their own bug (unless an admin unticks that box).
- Groups never *add* visibility; they only *restrict*. A bug in no group is open to all users.
- Changes to group controls affect existing bugs only after you confirm the on-screen question, so read it carefully.

4.4 How to file a bug

Click **New** (top of any page). If there are several products, pick one first.

The fields, in order

Field	What to put	Tip
Product / Component	Where the problem is	If unsure, pick the closest; a lead can move it
Version	The version you tested	Use `unspecified` if unknown
Severity	How bad: `blocker`, `critical`, `major`, `normal`, `minor`, `trivial`, `enhancement`	*Blocker* = nothing works / no workaround. *Enhancement* = a wish, not a bug
Priority	How urgent: `Highest` → `Lowest` (or P1–P5)	Usually set by the lead, not the reporter
Hardware / OS	Where it happened: `PC`, `Macintosh`; `Windows`, `Linux`, `Mac OS`, `All`	Pick `All` if it happens everywhere
Summary	One clear line: *what* is wrong, *where*	Good: "Login page: 'Forgot password' link opens a blank page". Bad: "login broken"
Description	The full story (see below)	The most important field
Assignee / QA contact / CC	Usually pre-filled from the component	Add people who should follow it
Target milestone	Which release should fix it	Leave to the lead
URL	A web address where the bug can be seen	
Keywords	Tags from a fixed list, e.g. `regression`, `crash`, `security`	Managed by admins
Depends on / Blocks	Numbers of related bugs	
Attachment	A screenshot, video, log or file	Tick **patch** if it is a code fix

Severity vs priority — the difference in one line: *severity* is how badly it hurts; *priority* is how soon we deal with it. A spelling mistake on the home page is low severity but can be high priority (everyone sees it). A crash in a feature nobody uses is high severity but low priority.

Writing a description that gets fixed fast

```
Steps to reproduce:
1. Open the app and log in as a normal user.
2. Go to Payments → New payment.
3. Enter amount 100 and tap Pay.

Expected result:
A confirmation screen appears.

Actual result:
The app closes immediately (crash). Happens every time.

Environment:
App version 2.3.1, Android 14, Samsung S23.
```

- **Steps** anyone can follow, numbered.
- **Expected** vs **actual** — the heart of every bug report.
- **Environment** — version, device, browser.
- **Attach proof** — a screenshot or short video says more than a paragraph.
- Before filing, **search for duplicates** (Bugzilla shows "possible duplicates" as you type the summary).

Click **Submit Bug**. You get a bug number; the assignee and CC list get an email.

After filing: what you can do on a bug

- **Add a comment** — questions, findings, progress. Comments cannot be edited or deleted (history is permanent), but can be marked **private** by insiders, or **tagged**.
- **Change fields** — anyone with `editbugs`; reporters can change their own bugs.
- **Add attachments** — files up to the size limit set by the admin (default about 1 MB, usually raised). Old files can be marked **obsolete**.
- **Flags** — small yes/no/question markers that can be attached to a bug or attachment and *requested from a person*, e.g. `review?` (please review this) → `review+` (approved) or `review-` (rejected). Admins define the flag types. **My Requests** lists flags waiting for you.
- **Votes** — if enabled, users can vote for bugs they care about; a bug with enough votes can confirm itself.
- **Time tracking** — estimated hours, hours worked, remaining, and a **deadline**. Shown to the `timetrackinggroup`.
- **See also** — links to related bugs in other bug trackers.
- **Personal tags** — private labels only you see, to organise your own bugs.
- **Clone this bug** — makes a copy pre-filled with the same details (handy when the same problem is in two products).

- **History** — the **View bug activity** link shows every change ever made.

4.5 How to search and filter bugs

Bugzilla has three ways to search, from simplest to most powerful.

1. Quick search (the box at the top of every page)

Type words and press Enter. Bugzilla looks in the summary, description and comments of open bugs.

- `login crash` — bugs mentioning both words.
- `1234` — opens bug 1234 directly.
- `"forgot password"` — the exact phrase.
- `ALL login` — include closed bugs too.
- `product:Website login` — only in the Website product.
- `:Payments` — only in the Payments component. `@alice@example.com` — assigned to Alice. `#regression` — with that keyword.
- `P1 critical` — priority / severity words are understood.

2. Simple search (Search → Simple Search)

Three boxes: **Status** (open / closed / all), **Product**, and **Words**. Good for most daily needs.

3. Advanced search (Search → Advanced Search)

A form with every field. Pick any combination:

- **Summary contains** (or *matches regular expression*, *doesn't contain* …).
- **Product, Component, Version, Milestone** — multi-select lists.
- **Status, Resolution** — e.g. all RESOLVED + FIXED.
- **Severity, Priority, Hardware, OS**.
- **Email addresses** — "assignee is / reporter is / CC contains" one or more people.
- **Dates** — *changed between* two dates, *bugs numbered*, *created within*.
- **Custom Search** (at the bottom) — build rules like *"Status Whiteboard contains `urgent`"* and *"Last changed is more than 30 days ago"*, joined with AND / OR / NOT.
- **Change History** — "the field *Status* changed to *RESOLVED* between these dates".

Click **Search**. The result is the **bug list** — a table you can **sort** by clicking a column, **change columns** on, and act on in bulk (**Change several bugs at once**).

Saving and sharing a search

- Under the results click **Remember search as…**, give it a name (*"My open P1 bugs"*). It appears as a link in the page footer.
- Searches can be **shared** with a group (Preferences → Saved Searches), so a whole team uses the same list.
- Every search is also just a **URL** — copy the address bar and send it to a colleague.

Browsing without searching

- **Browse** → pick a product → pick a component → open bugs listed.
- **My Bugs** (footer) → everything assigned to you or reported by you.

4.6 How email notifications work

Bugzilla emails people **when something changes on a bug they are connected to** — as assignee, reporter, QA contact, CC, or voter.

Switching email on (administrator, once)

1. **Administration → Parameters → Email**.
2. **mail_delivery_method** — `SMTP` (send through a mail server) or `Sendmail` (on Linux). `None` turns mail off; `Test` writes mails to a file instead of sending.
3. **smtpserver** — the mail server address, plus a user name / password and **smtp_ssl** if the server needs them.
4. **mailfrom** — the sender address people will see, e.g. `bugzilla@yourcompany.com`.
5. Save, then change any bug to test it.

Choosing what emails you (every user)

Preferences → Email Preferences shows a grid:

- **Rows** are events: a new bug is filed, status changes, the resolution changes, a comment is added, an attachment is added, priority/severity change, CC list changes, keywords change, a dependency changes, you are added or removed as assignee…
- **Columns** are your relationships to the bug: Assignee, Reporter, QA contact, CC, Voter, and "bugs I am watching" (Global).
- Tick the boxes for the combinations you want. For example: *email me about everything on bugs I own, but only new comments on bugs I am CC'd on*.
- Extra options: no email for changes **you** made yourself; emails only when the bug is **unconfirmed**; and so on.

Following other people's bugs

- Add yourself to a bug's **CC list** to receive its mails.
- **User watching** (same Preferences page): type a colleague's email to receive everything they receive — useful when covering for someone on holiday.

Scheduled summary emails ("Whining")

Whining sends you the results of a saved search on a timetable — for example *every Monday at 09:00, all my open bugs*; or *every day at 17:00, all unassigned critical bugs, to the lead*. Set it up under **Preferences → Whining** (users in the `bz_canusewhines` group; `bz_canusewhineatothers` to send to other people). The administrator must have scheduled Bugzilla's `whine.pl` to run every 15 minutes on the server.

Reminders about untouched bugs

The `whinedays` setting (Administration → Parameters) emails assignees about bugs that have sat in a new state for more than that many days.

4.7 How to generate reports and graphs

Click **Reports** at the top. There are three tools.

1. Current state — tabular reports

Count bugs **right now**, broken down by any field.

1. **Reports → Current State → Tabular report**.
2. Choose what to put on the **rows** (e.g. *Component*), the **columns** (e.g. *Severity*) and, optionally, a third axis for multiple tables (e.g. *Product*).
3. Narrow the data with the same filters as Advanced Search (e.g. only open bugs).
4. Click **Generate Report**. You get a table like "Login Page: 3 critical, 7 major, 12 normal". Every number is a link to that list of bugs. Export it as **CSV** for Excel.

2. Current state — graphical reports

Same as above, but drawn: a **bar chart**, **line chart** or **pie chart**. Pick the fields, pick the chart type, click **Generate Report**. Great for a slide in a status meeting: *open bugs by severity*, *bugs per component*, *who has the most bugs*.

3. Change over time — charts

Reports → Change Over Time → New Charts draws a line over days or weeks: *how many bugs were open each day this month?* Lines come from **data sets** — for each product, Bugzilla records the daily count per status and resolution (an administrator schedules `collectstats.pl` to run nightly). You can also chart any **saved search** to see, for example, whether "open P1 bugs" is going up or down. Combine lines, pick a date range, and save the chart.

Other useful views

- **Duplicates** (Reports → Duplicates) — the most frequently duplicated bugs: a list of the problems people hit most.
- **My Requests** — all flags waiting for your answer, and flags you asked others for.
- **Bug list → Change columns** — any search result can show the columns you want and be sorted; **Long format** prints every bug in full.
- **Dependency tree / graph** — on any bug, a picture of what blocks what.
- **Bug activity** — the complete timeline of one bug.

Keeping the system healthy (administrators)

- **Sanity Check** (Administration) finds and offers to repair broken links in the database — run it after big imports or upgrades.

- **Parameters** (Administration) holds every setting: the site address, email, attachment size limit (`maxattachmentsize`), which optional fields are on (`usetargetmilestone`, `useqacontact`, `usestatuswhiteboard`, `useclassification`), login rules, and more. Each has an explanation beside it.
- **Field Values** — add or rename severities, priorities, platforms, statuses and resolutions.
- **Bug Status Workflow** — a tick-box grid of which status may move to which, and whether a comment is required for that move.
- **Custom Fields** — add your own fields to every bug (a text box, a date, a drop-down list, a bug-number link…).
- **Flag types, keywords, classifications, milestones, versions** — each has its own page.
- **API keys** (Preferences → API Keys) — personal keys that let scripts act as you through the REST API, so automated tests can file bugs and reports can be pulled into other tools.
- **Backups** — Bugzilla keeps everything in its database; back that up regularly, plus the attachments folder if attachments are stored on disk.

Quick reference

A bug's journey: UNCONFIRMED → CONFIRMED → IN_PROGRESS → RESOLVED (FIXED / INVALID / WONTFIX / DUPLICATE / WORKSFORME) → VERIFIED. Reopen if it still fails.

Severity = how bad. **Priority** = how soon.

Product = the thing. **Component** = the part. **Version** = found in. **Milestone** = fix in.

Group = a named list of people. It can grant an *ability* (editbugs, canconfirm…) or *hide a product*.

Quick search tips: `1234` opens a bug · `"exact phrase"` · `ALL` includes closed bugs · `product:Name` · `:Component` · `@someone` · `#keyword`.

Get emails you want: Preferences → Email Preferences. **Get a weekly list:** Preferences → Whining. **See the big picture:** Reports → Tabular / Graphical / Charts.

End of guide.